Practice circuit 3

This is just a combination of the previous circuits, without any change. The program will be changed in a way that when the switch is pressed, it will toggle the state of the LED.

```
#!/usr/bin/python3
import RPi.GPIO as GPIO
import time
led = 8
ledstate = True

#this method will be invoked when the event occurs
def switchledstate(channel):
      global ledstate
      global led
      ledstate = not(ledstate)
      GPIO.output(led,ledstate)

swtch = 7
GPIO.setmode(GPIO.BOARD)
GPIO.setup(swtch,GPIO.IN, GPIO.PUD_DOWN)
GPIO.setup(led,GPIO.OUT,initial=ledstate)
# adding event detect to the switch pin
GPIO.add_event_detect(swtch, GPIO.BOTH, switchledstate, 600)
try:
      while(True):
            #to avoid 100% CPU usage
            time.sleep(1)
except KeyboardInterrupt:
      #cleanup GPIO settings before exiting
      GPIO.cleanup()
```

Pulse width modulation for analogue output

Pulse width modulation is a means to generate analogue signals from digital signal sources. In other words, PWM can fake analogue signals. PWM has two main characteristics – duty cycle and frequency. The duty cycle is the amount of time a signal is in the high state in one cycle. It is expressed in terms of a per cent measure. The frequency determines how fast the PWM completes a cycle. When the state of a digital signal toggles rapidly with changing duty cycle values, we feel that we get analogue signals.

There are two PWM pins in Raspberry Pi—*PWM0* and *PWM1*. The onboard analogue audio output uses both PWM channels. So it is advised not to use both simultaneously. In any case, the *RPi.GPIO* package facilitates only software PWM, which can be implemented on any GPIO pin. For this, the desired pin should be set as an output pin first. Then use:

```
pwm = GPIO.PWM(channel, frequency)
```

…where *channel* is the output pin and *frequency* is the frequency of your choice. Then start *pwm* with *pwm.start(dutycycle)* (the duty cycle value should be provided here). The duty cycle value can be changed using *pwm.ChangeDutyCycle(dutycycle)* and the frequency can be changed using *pwm.ChangeFrequency(frequency)*. At the end, stop *pwm* by using *pwm.stop()*.

Practice circuit 4

Let's brighten and dim an LED. The circuit is the same as Practice circuit 1.

```
import time
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
led=8
GPIO.setup(led, GPIO.OUT)
#starting with frequency 100
pwm = GPIO.PWM(led, 100)
#stating with 0, that is off state
pwm.start(0)
try:
      while 1:
            #duty cycle from 0% to 100%
            for dc in range(0, 101, 5):
            pwm.ChangeDutyCycle(dc)
            time.sleep(0.1)
            #duty cycle from 100% to 0%
            for dc in range(100, -1, -5):
            pwm.ChangeDutyCycle(dc)
            time.sleep(0.1)
except KeyboardInterrupt:
      pwm.stop()
      GPIO.cleanup()
```

This article would not be complete without mentioning analog inputs. Since Raspberry Pi is a digital device without any built-in ADC (analogue-to-digital converter), reading analogue data from sensors or circuits requires an external ADC. The commonly used ADC is MCP3008. Discussing this ADC and its SPI communication protocol is beyond the scope of this article. Raspberry Pi supports SPI, I²C and UART protocols. I have found UART with Arduino as the ADC to be the simplest option of the three.

RPi.GPIO is just one package for GPIO programming. There are other packages like *gpiozero* available. Have fun making things!

All the programs discussed here are available at *gitlab.com/noblephilip/rpi*.

By: Noble Philip

The author is a free software enthusiast and a Raspberry Pi developer. He has more than 10 years of programming experience. At present, he works at Amal Jyothi College of Engineering, Kerala. He can be contacted at *noblephilip(at)outlook(dot)com*.